Deploying the Cluster (Day-1)

Kubernetes Production · Lesson 6

Bringing up a production cluster with **Kubespray**

`kubespray.io` · `kubernetes-sigs/kubespray`

Learning Objectives

By the end of this lesson you will be able to:

- Run the **Day-1 deploy** with `ansible-playbook … -b cluster.yml`
- Explain why `-b` / `--become` is required and what **idempotency** buys you
- Describe what `cluster.yml` does **end-to-end** (preflight → add-ons)
- Add API SANs with `supplementary_addresses_in_ssl_keys`
- Understand that **Kubespray runs kubeadm** under the hood — the bridge
- **Retrieve access** (`kubeconfig_localhost`, `artifacts_dir`, `admin.conf`)
- **Verify** the cluster and use `--limit` for partial runs

> This lesson **replaces** the manual `kubeadm` deploy from the CKA course — same cluster, driven declaratively.

6.1 The Deploy Command

One Command Brings Up the Cluster

The whole Day-1 bring-up is a **single, idempotent** `ansible-playbook` run against the inventory built in Lesson 2:

```
ansible-playbook -i inventory/mycluster/hosts.yaml -b cluster.yml
```

- `-b` / `--become` = privilege escalation (sudo → root). **Required** — without it the run fails on the first task (cannot install packages, write to `/etc/kubernetes`, restart the kubelet).
- A full first run takes **~20 minutes** (longer on slow networks / many nodes).
- **Idempotent**: re-running **converges** the cluster to the declared state — it does **not** rebuild from scratch, so it is safe to retry.

> CKA tip: know this one-liner — and *why* `-b` is mandatory — cold.

The Flags That Matter

```
ansible-playbook -i inventory/mycluster/hosts.yaml -b \
  -u $USERNAME --private-key=~/.ssh/id_rsa cluster.yml
```

Flag	Meaning
`-i <inventory>`	Inventory with node definitions & groups (`hosts.yaml`)
`-b` / `--become`	**become** — sudo to root. **Required**
`-u $USERNAME`	SSH login user on the targets
`--private-key=…`	SSH private key to reach the nodes
`-v` / `-vv` / `-vvv`	Verbose — increase to debug failing tasks
`--limit <host|group>`	Restrict the run to a subset of hosts
`--extra-vars "k=v"`	Override variables on the command line

> The playbook may be written `cluster.yml` or `playbooks/cluster.yml`; the inventory `hosts.yaml` or `inventory.ini` — same playbook, same flags.

6.2 What `cluster.yml` Does

The Deploy Pipeline (End-to-End)

The single `cluster.yml` playbook orchestrates the whole bring-up, in order:

1. **Preflight** — validate inventory, variables, host reachability; fail fast
2. **bootstrap-os** — OS prep on every node: packages, Python, kernel modules, sysctls, **disable swap**
3. **Install runtime + binaries** — container runtime (containerd) plus `kubeadm` + `kubelet` + `kubectl`
4. **Run kubeadm** — `kubeadm init` the first control plane, `kubeadm join` the rest (etcd clustered on the `etcd` group)
5. **CNI** — deploy the network plugin (**Calico** by default)
6. **Add-ons** — optional components from `group_vars` (metrics-server, ingress…)

> Same outcome as the manual CKA path — just one declarative pass instead of per-node imperative steps.

API SANs: `supplementary_addresses_in_ssl_keys`

To reach the API server through an address **not** already known — an external IP, a load-balancer VIP, or a DNS name — add it to the cert SANs **before** deploying, or `kubectl` from outside fails TLS verification:

```
# group_vars/k8s_cluster/k8s-cluster.yml
supplementary_addresses_in_ssl_keys:
  - 197.140.32.10      # external VIP / load balancer
  - kube.example.com   # DNS name
```

- This is the declarative equivalent of kubeadm's `--apiserver-cert-extra-sans` — **same X.509 SANs**.
- Adding a SAN **after** the cluster is up means **re-issuing certificates** — set it up front.

6.3 Under the Hood: Kubespray Runs kubeadm

Kubespray Drives — kubeadm Is Still the Engine

Kubespray does **not** reinvent the bootstrap. Step 4 of `cluster.yml` calls **`kubeadm init`** on the first control-plane node and **`kubeadm join`** on every other node — so the same ordered **kubeadm phases** run *inside* the playbook:

```
preflight → certs → kubeconfig → kubelet-start → control-plane
→ etcd → upload-config → mark-control-plane → bootstrap-token → addons
```

- Certs, the **control-plane taint**, bootstrap **tokens**, the `/etc/kubernetes` layout — **all identical** to the manual path.
- Your **kubeadm knowledge still applies**, and the **CKA exam may ask kubeadm directly**.

> Kubespray is the **driver**; kubeadm is the **engine**. You just drive it declaratively.

Manual kubeadm vs Kubespray

Manual CKA (the old Lesson 2)	Kubespray (this lesson)
Run `kubeadm init` / `kubeadm join` by hand	The playbook runs them for you
`kubectl apply` a CNI manifest	`cluster.yml` installs the CNI (Calico)
Copy `admin.conf` yourself	`kubeconfig_localhost: true` fetches it
Per-node, imperative, error-prone	Declarative, **idempotent**, repeatable

> Same engine, same artifacts — different ergonomics. Learn kubeadm for the exam; use Kubespray in production.

6.4 Retrieving Access (kubeconfig)

Let Kubespray Fetch It

The admin kubeconfig lives at `/etc/kubernetes/admin.conf` on the control plane.

Easiest path — set these **before** the run:

```
# group_vars/k8s_cluster/k8s-cluster.yml
kubeconfig_localhost: true   # copy admin.conf into the artifacts dir
kubectl_localhost: true      # also drop a kubectl binary + kubectl.sh wrapper
```

After deploy, files appear under **`artifacts_dir`** (default `inventory/mycluster/artifacts/`):

```
cd inventory/mycluster/artifacts
export KUBECONFIG=$PWD/admin.conf
./kubectl.sh get nodes
```

Or Copy `admin.conf` Manually

Take ownership on the controller, copy it, and point the `server:` URL at the **reachable** controller address:

```
ssh $USERNAME@$IP_CONTROLLER_0 \
  "sudo chown $USERNAME:$USERNAME /etc/kubernetes/admin.conf"

scp $USERNAME@$IP_CONTROLLER_0:/etc/kubernetes/admin.conf ./admin.conf
# edit the 'server:' field → external/reachable controller IP, then:
export KUBECONFIG=$PWD/admin.conf
```

> If `server:` points at an address **not** in the cert SANs, you get a TLS error — exactly what `supplementary_addresses_in_ssl_keys` prevents.

6.5 Verifying the Cluster

Verify the Cluster

```
kubectl get nodes          # every node should be Ready
kubectl get pods -A        # CNI, CoreDNS, kube-proxy… all Running
kubectl get pods -n kube-system
kubectl top nodes          # works once metrics-server add-on is enabled
```

- A node sits **NotReady** until the **CNI** pods are up — same as the manual kubeadm path.
- `cluster.yml` installs the CNI for you, so after a clean run **all nodes should be Ready**.

> CKA tip: if nodes are NotReady, check `kubectl get pods -n kube-system` — the network plugin pods are the usual suspect.

6.6 Partial Runs with `--limit`

Scope a Run with `--limit`

Use `--limit` to target a subset of hosts — retry a single node that failed, or act on one group without touching the rest:

```
# retry just one node after a transient failure
ansible-playbook -i inventory/mycluster/hosts.yaml -b cluster.yml \
  --limit node4

# act on a whole group
ansible-playbook -i inventory/mycluster/hosts.yaml -b cluster.yml \
  --limit kube_node
```

> Because the playbook is **idempotent**, a limited re-run only fixes what drifted on the targeted hosts — the rest of the cluster is untouched.

Key Takeaways

- Day-1 is **one idempotent command**: `ansible-playbook -i inventory/mycluster/hosts.yaml -b cluster.yml` (~20 min)
- `-b` / `--become` **is required** — every task needs root
- `cluster.yml` runs **preflight → bootstrap-os → runtime+kubeadm/kubelet/kubectl → kubeadm → CNI → add-ons**
- **Kubespray runs kubeadm** — the kubeadm **phases** happen inside the playbook, so kubeadm knowledge (and CKA questions) still apply
- Get access via `kubeconfig_localhost` / `kubectl_localhost` (`artifacts_dir`) or by copying `admin.conf` and fixing `server:`
- Add external/LB/DNS endpoints to `supplementary_addresses_in_ssl_keys` before deploying
- Verify with `kubectl get nodes` / `get pods -A`; use `--limit` to retry one node or group

End of Lesson 6

Next: Lesson 7 — High Availability

`ansible-playbook -i …/hosts.yaml -b cluster.yml` · `--limit` · `kubeconfig_localhost: true` · `kubectl get nodes`